

テキサスポーカー自動対戦時のCPU

1 プログラムについて

quot HvsH.py が人対人のためのプログラム、quot CvsC.py が CPU 対 CPU のプログラムとなっている。quot simulation.py が 10000 回の分析の際に用いたプログラムである。人対人に関しては、操作が表示されるのでその操作に従い、操作ができる。またそれぞれの exe ファイルを開いていただくと、プログラムが実行される。終了する際には q を押すと終了することが可能である

2 プログラムの説明

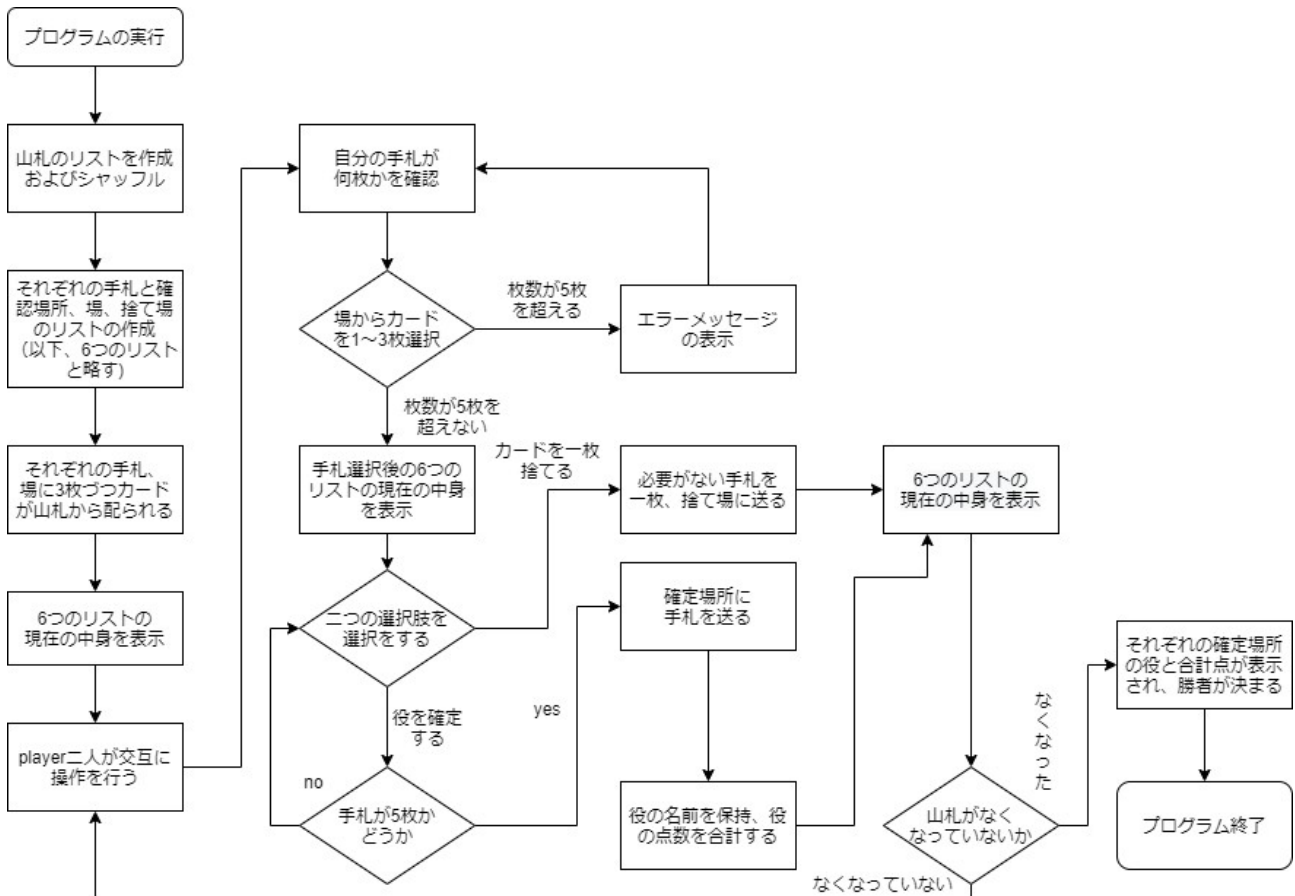


図 1: プログラムのフローチャート

2.1 プログラムの手順

1. 山札のリストを作成、およびシャッフル
2. それぞれの手札と確定場所、場、捨て場のリストを作成 (以下 6 つのリストと略す)
3. それぞれの手札、場に 3 枚ずつカードが配られる
4. 6 つのリストの現在、保持しているカードのマークと数字を表示
※リストとは、要素 (数字、文字) を含んだ配列のことである
5. 以下 player が交互に操作を行う
6. 手札が 5 枚を超えないように場から 1 3 枚必要なカードを選択
7. 6 つのリストの保持しているカードのマークと数字を表示

8. 手札のから役の確定場所に移動もしくは手札から 1 枚カードを捨て札に移動
(手札のカードが 5 枚でないと役の確定は行えない)
9. 6 つのリストの保持しているカードのマークと数字を表示
10. 山札なくなっていないとき、player を交代し、5 に戻る
11. 山札がなくなったときにゲームが終了それぞれの確定場所の役、合計点が表示され勝者が決まる

3 CPU

CPU の組み込んだ際のプログラム手順の 6,8 を担当する CPU の動きを簡単に説明する。

表 1：7 つのパターン

パターン	説明
1	場の左から手札に入れる (一枚引く)
2	場の真ん中から手札に入れる (一枚引く)
3	場の右から手札に入れる (一枚引く)
4	場の左と右から手札に入れる (二枚引く)
5	場の真ん中と右から手札に入れる (二枚引く)
6	場の右と真ん中から手札に入れる (二枚引く)
7	3 枚すべてを手札に入れる

3.1 CPU の流れ

1. カードを引くパターンを 7 つに分割する
 - ・カードのパターンについては、表 1 を参考いただきたい。
2. 手札の要素を新しくつくった別のリストにコピーする
 - ・作成する理由としては、別のリストにコピーすることでコピー元である手札に影響が出ないようにするためである。(以下、仮想手札と呼ぶ)
3. 仮想手札で 7 つのパターンすべてを試す
 - ・試すとは実際に仮想手札に場からカードを追加することである。
4. 今持っているカードの最善手を導き出すために評価関数を用いる。
 - ・今回でいう評価関数とは、最善手を導き出すためのものであるパターン一つ一つを仮想手札で再現し、パターンの状況で揃う役を点数が高い順に優先順位をつけ、その優先順位によってどのパターンを採用するかを決めるというものである
5. パターンごとにつけられた優先順位が一番高いものが採用され、元の手札に代入される
6. 対戦手順 3 における一枚捨てるか、確定するか処理においては 7 つのパターンを試す際に仮想手札に 5 枚あり役がそろっている (豚以外を指す) 場合に確定、仮想手札が 5 枚ではなく、5 枚であっても豚である場合には、一枚捨てる
7. 一枚捨てる場合において、捨て札の決定にも評価関数を用いる
 - ・持っている仮想手札を一枚ずつ捨てたときに、一番役に近づいたものを優先順位の高いものとする (確定できる手札のほうが優先順位は高くなる)

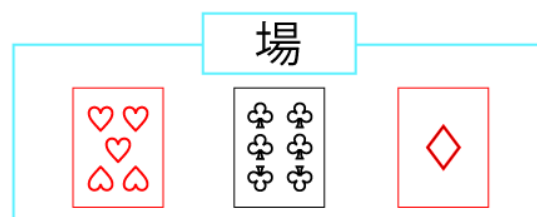
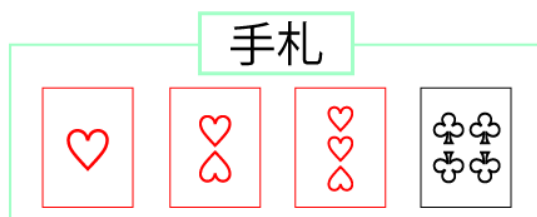
表 2 : 優先順位

優先順位	役もしくは状態	仮想手札 (枚)	仮想手札が 5 枚でかつ豚かどうか
1	ロイヤルストレートフラッシュ	5	×
2	ストレートフラッシュ	5	×
3	フォーカード	5	×
4	フルハウス	5	×
5	フラッシュ	5	×
6	ストレート	5	×
7	スリーカード	5	×
8	ツーペア	5	×
9	ワンペア	5	×
10	スリーカードが揃っている	4	×
11	ワンペアが揃っている	4	×
12	ワンペアが揃っている	3	×
13	マークが 4 枚揃っている (フラッシュに近い)	5	○
14	マークが 3 枚揃っている (フラッシュに近い)	5	○
15	マークが 3 枚揃っている (フラッシュに近い)	4	×
16	数字が連続で 3 つ揃っている (ストレートに近い)	4	×
30	豚	3	×
31	豚	4	×
32	豚	5	○

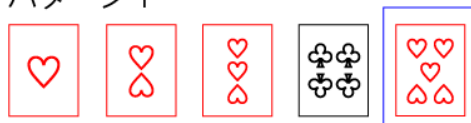
※仮想手札が 5 枚の時に豚である時もしくは、4 枚以下の時は
仮想手札から一枚捨てた時の状態で優先順位をつける

3.2 例

 は仮想手札に追加したカード

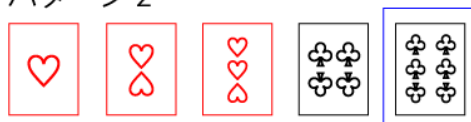


パターン 1



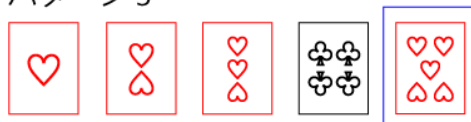
「ストレート」優先順位は 6

パターン 2



豚であるため一枚捨てた場合を考える
一枚ずつ捨てた場合を考えると、フラッシュ
に近いといえるマークが 3 枚揃っている時が
あるため優先順位は 14、捨てるカードは先に
優先順位を 14 と判断する♣4 が選ばれる

パターン 3

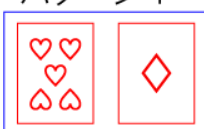


「ワンペア」優先順位は 9

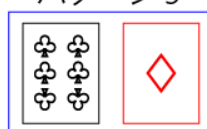
パターン 4 から 7 までは仮想手札が 5 枚を超えるため除外

下記がパターン 4 から 7 までの仮想手札に追加したカード

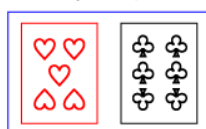
パターン 4



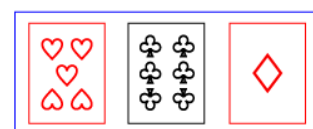
パターン 5



パターン 6



パターン 7



上記のような場合パターン 4 からパターン 7 までは、仮想手札が 5 枚を超えてしまい、ルールにのっとっていないため除外する。パターン 1 の場合、仮想手札で試すとストレートができる。パターン 2 の場合 5 枚ではあるが豚であるため、一枚捨てる場合で考え、揃っていないがフラッシュに近いとなる。ここで捨てる札を選び、今回の場合には クローバーの 4 を捨てる。パターン 3 の場合には数字の 1 でワンペアができる。これら 3 つのパターンの優先順位を考えると、フラッシュに近いが豚であるパターン 3 の優先順位が一番低く、高い点数であるストレートが揃うパターン 1 の優先順位が高くなる。そのため、今回の手札においては、パターン 1 の場合の仮想手札が元の手札に代入され確定の処理を行う。このように CPU が処理を行っている。

4 分析

4.1 課題 1

「Quot」のルールの中で CPU が自分の手札の役作りに専念したプログラムを作成、CPU 同士で対戦する。今回このシミュレーションを 10000 回を行い以下の検証を行った。

- 先攻の CPU と後攻の CPU の勝率の比較
- 対戦手順 3 における選択をどちらにしたかの回数
- それぞれの平均の得点

CPUvsCPUの戦績

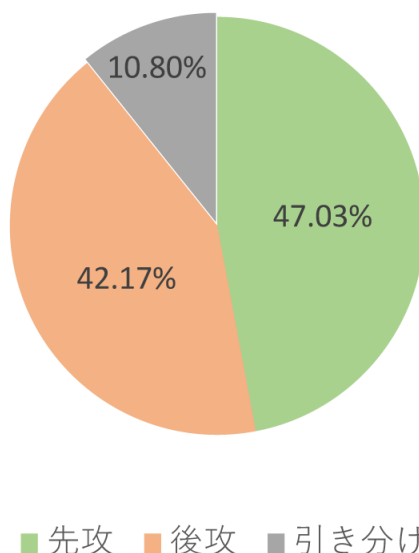


図 2 : CPUvsCPU の戦績

表 3 : CPUvsCPU の戦績、平均の得点

	先攻	後攻
勝利数	47.03%	42.17%
引き分け	10.80%	
平均得点	16.23 点	15.91 点

戦績の結果としては、先攻が 47.03%、後攻が 42.17%、引き分けが 10.80% と先攻の CPU が 4.89% 上回った。また得点が先攻のほうがわずかにではあるが高い。これらのことから実装した CPU 同士を「Quot」で戦わせると先攻の勝率がよいことがわかる。それはなぜなのか、私は、単純に先に出しているからだと考える。今回の AI ではカードが揃っていない時、5 枚だが豚である時にカードを 1 枚ずつ引いていく。役が揃うまでは同じ行動をするため、行動回数が多いほうが有利に働く。下記の表 3 のデータを見ていただいてわかるように行動回数の平均が先攻のほうが多いことがわかる。その結果として、先攻の勝率がよいという結果に至ったのだと考える。

表 4：対戦手順 3 における選択をどちらにしたかの平均回数

	先攻	後攻
選択 1 を選んだ平均回数	7.9 回	7.41 回
平選択 2 を選んだ平均回数	3.25 回	3.15 回

- 選択 1 とは、対戦手順 3 で一枚カードを捨てること
- 選択 2 とは、対戦手順 3 で確定を行うこと

表 2 において、平均得点があまり高くないことから組み込んだ CPU の戦略について読み取ることができる。そこからそれぞれの CPU が対戦手順 3 における選択をどちらにしたかの平均回数を出すことでよりデータを集めた。Quot には、様々な戦略が存在すると考えるが大きく分けると 2 つにわけることができる。一つが手札、こまごまと確定していき点数を稼ぐタイプ、もう一つが高い役が揃うまでカードを捨て続け、逆転を狙うタイプである。今回の CPU は表 2 の平均点、表 3 の対戦手順 3 における選択をどちらにしたかの平均回数から二つ目のタイプで動いていることがわかる。しかし予想以上に慎重になってしまっており、平均のターン数が約 11 ターンのところ、それぞれの 1 試合で平均 70% の選択を一枚捨てることにしている。一枚捨てる選択を何回も連続で行ってしまうと、一枚しか変えることができないため運任せになってしまう。少ないターン数で必要なカードを引くことができればよいが、できなかったときに組み込んだ CPU のように点数があまり伸びない。そのため、今の手札で役を作ることを何ターンかで辞めさせる処理があれば、より高い点数をとる可能性があると考ええる。